402 SWEG Dashboard
Week 1 Deliverables

1. Project Charter

Team Roles
Conner - Scrum Master, Sprint 1
Olivia - Scrum Master, Sprint 2
Molly - Scrum Master, Sprint 3
Ritvik - Developer
Taylor - Developer

Scrum Master Rotation

Sprint	Dates	Scrum Master	Key events they run
1. Discovery and metrics	Thu Oct 29 to Sun Nov 8	Conner	Charter, survey draft, Sprint 1 review prep (week of Nov 9)
2. Design and prototype	Mon Nov 9 to Sun Nov 22	Olivia	Survey analysis, prototype build, Sprint 2 review (Nov 23 or 24)
3. Test and refine	Mon Nov 23 to Thu Dec 3	Molly	Testing around Thanksgiving, final report, presentation

Five Project Risks

1. Selection of misleading KPI's
2. Limited access to real performance data
3. Scope creep
4. Poor team coordination
5. Limited usability testing

Kickoff Interview Questions

Business Goals and Decision-Making

1. What are the three most important decisions leadership hopes to make using this dashboard?
2. What performance problems or inefficiencies are currently difficult to identify?
3. How would you define success for the dashboard six months after implementation?

Existing Metrics and Data Sources

4. Which performance metrics does 402 SWEG currently track, and how are they collected?
5. What software tools or systems contain the performance data we would need?
6. Are there important performance indicators that leadership wants to track but currently cannot measure?

KPI Selection

7. Which matters most when evaluating team performance: delivery speed, software quality, system stability, or team health?
8. How should the dashboard distinguish genuine improvements in productivity from increases in activity that do not improve outcomes?

Dashboard Design and Usability

9. What information should leadership see immediately upon opening the dashboard?

10. Should users be able to compare different teams, time periods, or projects? If so, which comparisons would be most valuable?

11. How frequently would managers and team leaders realistically review this information?

Constraints and Expectations

13. Are there data security, confidentiality, or access restrictions we should consider?
14. What would make a dashboard unsuitable or unsuccessful from your perspective?
15. If development time becomes limited, which dashboard capabilities should we prioritize above everything else?

Dashboard User Research Survey Questions

1. What is your primary role within the organization?
Multiple choice

- Project Manager
- Team Leader
- Developer
- Other (please specify)

2. How frequently do you review your team's software development performance?
Multiple choice

- Daily

- Several times per week
- Weekly
- Monthly
- Less frequently

3. Which performance indicators are most important when evaluating your team's effectiveness?
Select up to three

- Sprint progress/burndown
- Deployment frequency
- Lead time for changes
- Defect rates/software quality
- CI/CD pipeline success rate
- Unit test coverage
- System stability/recovery time
- Team collaboration and well-being
- Other (please specify)

4. What is your biggest challenge when evaluating team performance today?
Multiple choice

- Information is spread across multiple systems
- Available metrics do not accurately represent performance
- Performance trends are difficult to identify
- Reports take too long to prepare
- Data is incomplete or outdated
- Other (please specify)

5. Which decisions would you most like a performance dashboard to support?
Select up to three

- Identifying development bottlenecks
- Improving software quality
- Evaluating sprint progress
- Allocating team resources
- Identifying performance trends
- Improving development processes
- Other (please specify)

6. How useful would it be to compare team performance across different time periods?
Five-point scale

- 1 — Not useful
- 2 — Slightly useful
- 3 — Moderately useful

- 4 — Very useful
- 5 — Extremely useful

7. Which dashboard presentation format would be most useful for your work?
Multiple choice

- Summary cards with current KPI values
- Line charts showing performance trends
- Bar charts comparing teams or projects
- Detailed tables with numerical data
- A combination of summary cards and interactive charts

8. What device would you primarily use to access the dashboard?
Multiple choice

- Desktop computer
- Laptop
- Tablet
- Smartphone

9. How concerned are you that performance metrics could be misinterpreted or encourage teams to focus on improving numbers rather than actual outcomes?
Five-point scale

- 1 — Not concerned
- 2 — Slightly concerned
- 3 — Moderately concerned
- 4 — Very concerned
- 5 — Extremely concerned

10. What information or dashboard capability would most improve your ability to manage team performance?

Schedule

Main meetings: during class time

Backup meeting: Sundays at 9:00 PM, virtual (kept on calendar, can be canceled)

Standups: start of each class during a sprint

Sprint planning: first class of each sprint

Retrospectives: end of Sprints 1 and 2

Sprint Dates

Sprint 1, Discovery and Metrics: Oct 29 to Nov 8

Sprint 2, Design and Prototype: Nov 9 to Nov 22

Sprint 3, Test and Refine: Nov 23 to Dec 3

Weekly deliverables: Sundays, 11:59 PM

Final deliverables: Thursday, Dec 3, before class

Team Agreement

1. All work is tracked on our dashboard(Trello equivalent), with an owner and due date for every task.

2. We communicate by group text and reply within 24 hours.

3. If you're stuck or behind, say so early. The team helps.

4. Missed work is handled privately by the Scrum Master first, with no public call-outs.

5. New ideas go to the backlog and get decided at sprint planning.

6. Outgoing Scrum Masters hand off the backlog and blockers to the next one.

7. Everyone presents part of the final presentation.

Scope

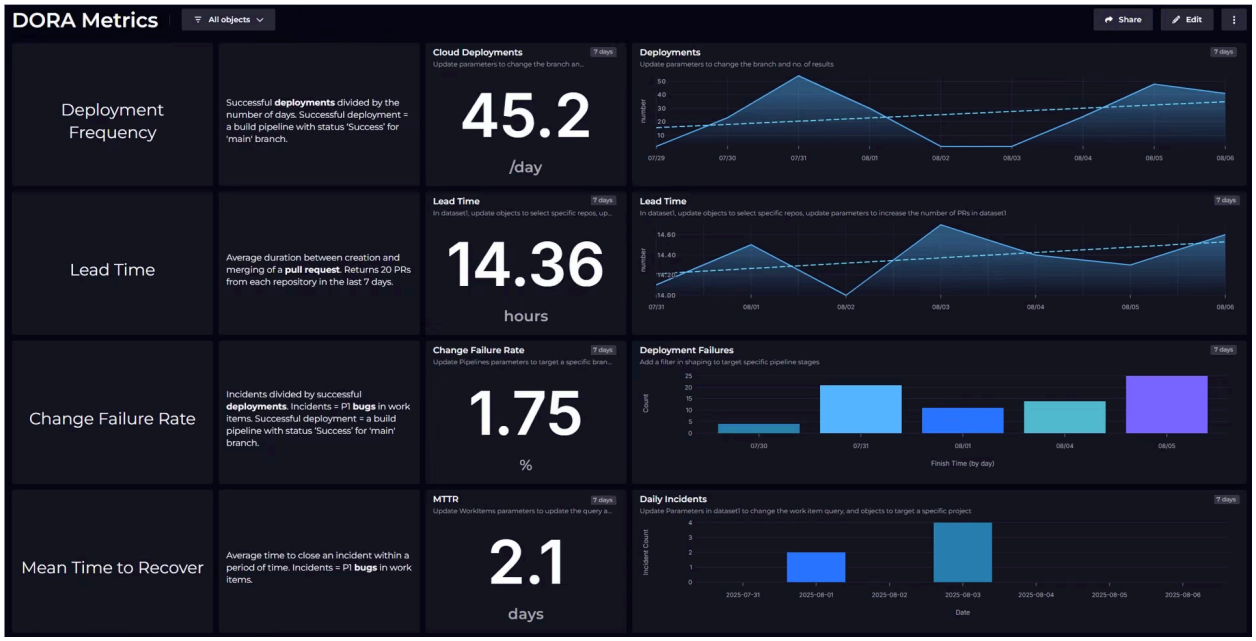
In: team-level KPIs, research, personas, journey map, Figma(or Figma equivalent) prototype with sample data, usability testing, final report, presentation

Out: individual developer metrics or rankings, real data connections, coded dashboard

Comparable Dashboards and KPIs

We'd like your quick reactions before we design the 402 SWEG dashboard. Below are three real dashboards, the DORA and SPACE research behind our 15 candidate metrics, a few design choices, and seven questions.

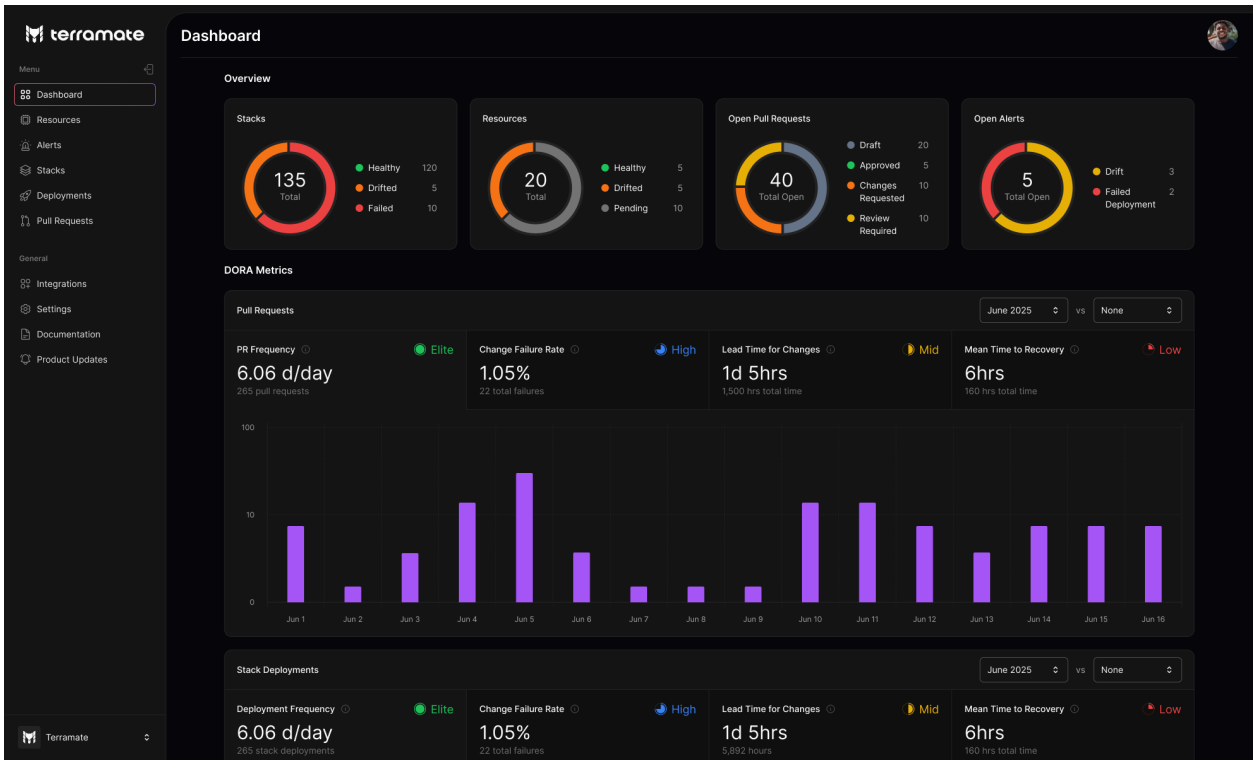
Example A: SquaredUp DORA dashboard (Azure DevOps data)



Our assessment

Criterion	What we see
Metrics shown	DORA 4: deployment frequency (45.2/day), lead time (14.36 hrs), change failure rate (1.75%), mean time to recover (2.1 days)
How trends show	7-day line with a dashed trend line; daily bars for failures and incidents
Read speed	Fast: name, definition, number and trend sit in one row
Borrow	A definition beside every number. Definitions tied to Azure DevOps data (a successful pipeline on main; an incident = a P1 bug work item)
Avoid	Only 7 days of history. No target, so 45.2 isn't marked good or bad. Bar colors carry no meaning. Lead time counts only pull-request open to merge

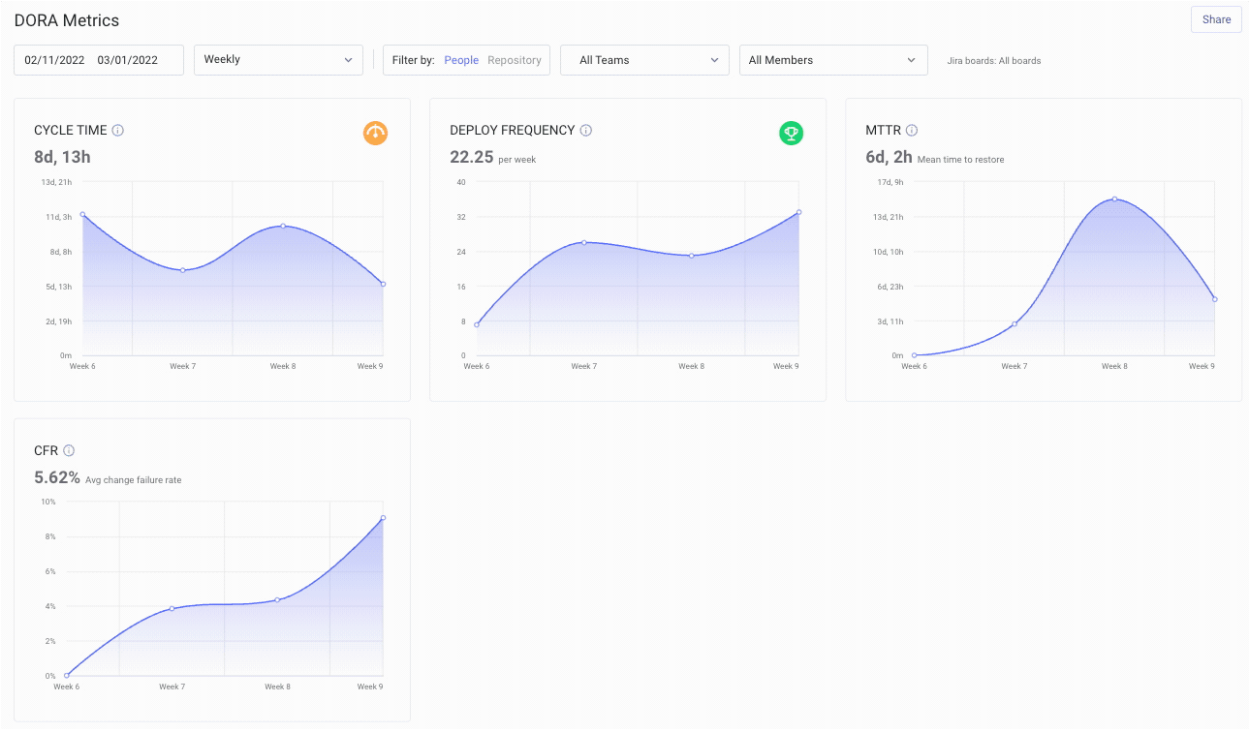
Example B: Terramate dashboard



Our assessment

Criterion	What we see
Metrics shown	DORA 4, each tagged Elite / High / Mid / Low. Status donuts for healthy, drifted and failed items. Open pull requests by state
How trends show	Month picker with a "vs" comparison period; daily bars underneath
Read speed	Very fast: the tier word answers "good or bad?" before you read the number
Borrow	Tier label shown as a word and an icon, not color alone. A "vs" period comparison. A context line under each number ("22 total failures")
Avoid	Donuts are hard to compare. Industry DORA tiers may not suit flight software. The overview tiles are infrastructure-specific

Example C: LinearB DORA metrics



Our assessment

Criterion	What we see
Metrics shown	Cycle time (8d 13h), deploy frequency (22.25/week), mean time to restore (6d 2h), change failure rate (5.62%)
How trends show	Smoothed weekly area charts over 4 weeks
Read speed	Medium: clean, but each chart has its own scale and there are no definitions
Borrow	Date, interval and team filters at the top. A badge that flags a metric against a benchmark. Minimal, light layout
Avoid	"Filter by People" and "All Members" allow individual views, which breaks your team-level rule. Smoothed curves overstate swings (failure rate going 0% to 9% looks dramatic). No target lines

KPI research: DORA and SPACE

DORA (Google's DevOps Research and Assessment)

- **Deployment frequency:** how often working software is released
- **Change lead time:** time from code written to code running

- **Failed deployment recovery time:** how fast a team recovers when a release breaks
- **Change fail rate:** % of releases that cause a problem
- **Deployment rework rate:** % of releases that were unplanned fixes

SPACE

- **Satisfaction and well-being** (team pulse survey)
- **Performance** (escaped defects, change fail rate)
- **Activity** (deployments, completed work)
- **Communication and collaboration** (code review turnaround)
- **Efficiency and flow** (cycle time, unplanned work)

What this means for 402 SWEG

- Pair every speed metric with a quality metric so fast but broken never looks good
- Team level only, never individuals
- Data comes from Azure DevOps and GitLab
- "Deployment" needs a local definition since software goes onto aircraft, not a website
- Quality and stability matter more than raw speed for flight software

Checklist for every KPI

1. What decision does it help someone make?
2. Is it team level?
3. What metric balances it?
4. Can Azure DevOps or GitLab produce it?
5. Does it hold up for aircraft software?

Candidate metrics: include or not?

These 15 candidates come from DORA, SPACE, DoD software metrics guidance and your own examples. All are measured per team, never per person. Lines of code is left out on purpose, as you suggested.

#	Metric	What it measures	Area	Data source
1	Delivery frequency	Builds delivered to the lab or test, per week	Speed	Pipelines
2	Lead time for changes	Commit to delivered build, in days	Speed	GitLab / Azure DevOps
3	Cycle time by stage	Time in code, review, test and delivery: where work gets stuck	Speed	Work items, merge requests
4	Work in progress	Open work items per team	Flow	Boards

5	Sprint commitment / burndown	Planned vs. completed work each sprint	Predictability	Boards
6	CI/CD pipeline pass rate	% of pipeline runs that pass	Quality	Pipelines
7	Unit test coverage	% of code covered by automated tests, and its trend	Quality	Coverage reports
8	Escaped defects	Bugs found in flight test or the field vs. in-house	Quality	Bug work items
9	Change failure rate	% of deliveries that need a fix or rollback	Stability	Pipelines, bugs
10	Recovery time	Time to fix a failed delivery or a broken pipeline	Stability	Pipelines, bugs
11	Rework rate	% of work that is unplanned bug fixing	Stability	Work items
12	Vulnerability fix time	Days to fix critical and high security scan findings	Security	Security scans
13	Code review turnaround	Hours from review request to approval	Collaboration	Merge requests
14	Work mix	% of effort on new capability vs. bug fixes vs. maintenance	Value	Work item types
15	Team health pulse	Quarterly team survey score, shown only as a team average	Team health	Survey

Anything missing that leadership already asks about?

Questions for Dexter

#	Question	Your answer
1	Think of the last time a team fell behind or shipped a quality problem. How did leadership find out, how late was it, and what signal would have warned you earlier?	

2	Between a developer finishing a change and that change running on an aircraft, what are the major steps? Which step takes the longest, and which are outside the team's control? Which step should count as a "delivery" for measuring?	
3	How much history should the dashboard cover (last sprint, quarter, year)? Roughly how many teams, projects and pipelines would it pull from, and how many teams run agile sprints vs. traditional plans?	
4	Do teams use Azure DevOps and GitLab side by side, or does each team use one or the other?	
5	Who is "leadership" for this dashboard: squadron, group, or directorate level?	
6	Where are bugs from flight test or the field logged?	
7	What does leadership see today (slides, Art of the Possible boards, spreadsheets)?	
8		

Thank you! Any other thoughts are welcome as comments anywhere in this doc.